

POKAZNA VEŽBA 8

Projektovanje procesora

Potrebno predznanje

- Razumevanje kombinacionih mreža, sekvencijalnih mreža, automata
- Arhitektura računara i assembler

Šta će biti naučeno tokom izrade vežbe?

Nakon urađene vežbe, bićete u mogućnosti da:

- Kombinujete standardne kombinacione i sekvencijalne komponente, koje su same po sebi veoma jednostavne, u veoma složeni sistem čija primena značajno prevazilazi primenu pojedinačnih komponenti
- Implementirate komponente procesora
- Projektujete složene sisteme sastavljene iz nekoliko modula
- Simulirate složeni digitalni sistem i razumete njihove simulacione dijagrame
- Napravite vezu između znanja iz programske podrške i fizičke arhitekture digitalnih sistema
- Razumete kako funkcioniše procesor

Apstrakt i motivacija

Mozak vašeg ličnog računara je **procesor**, koji predstavlja univerzalnu strukturu za računanje koja može da izvršava složene aritmetičke i logičke operacije. U ovoj vežbi projektovaćete složenu strukturu za računanje, preteču procesora, koja može da izvršava osnovne operacije – sabiranja, oduzimanja, uvećanja, umanjevanja, logičke operacije i pomeranja. Naučićete kako da koristite multipleksere za izbor operanada, kako da projektujete aritmetičko-logičku jedinicu, kako da odredite statusne bite (*zero*, *carry*, *sign*) i kako da koristite registre za čuvanje izračunatih vrednosti. Konačno, implementiraćete i upravljačku jedinicu koja upravlja svim komponentama u procesoru.

TEORIJSKE OSNOVE

1. Arhitektura procesora

Kako bi programer napisao program koji može da se izvrši na nekom procesoru, treba da poznaje tipove podataka, formate instrukcija, instrukcijski set, način adresiranja, dostupne registre, itd. Upravo **arhitektura računara** obuhvata sve ono što je potrebno znati o računarskom sistemu, kako bismo bili u mogućnosti da pišemo i izvršavamo programe. Za izvršavanje programa zadužen je hardver računara i njegove osnovne komponente su:

- **Memorija** – čuva podatke i program (instrukcije)
- **Procesor** – obavlja obradu podataka izvršavanjem programskih instrukcija
- **Ulazno/izlazni uređaji** – omogućavaju unos ulaznih podataka, kao i prikaz rezultata obrade
- **Magistrala** – povezuje i razmenjuje podatke između komponenti sistema

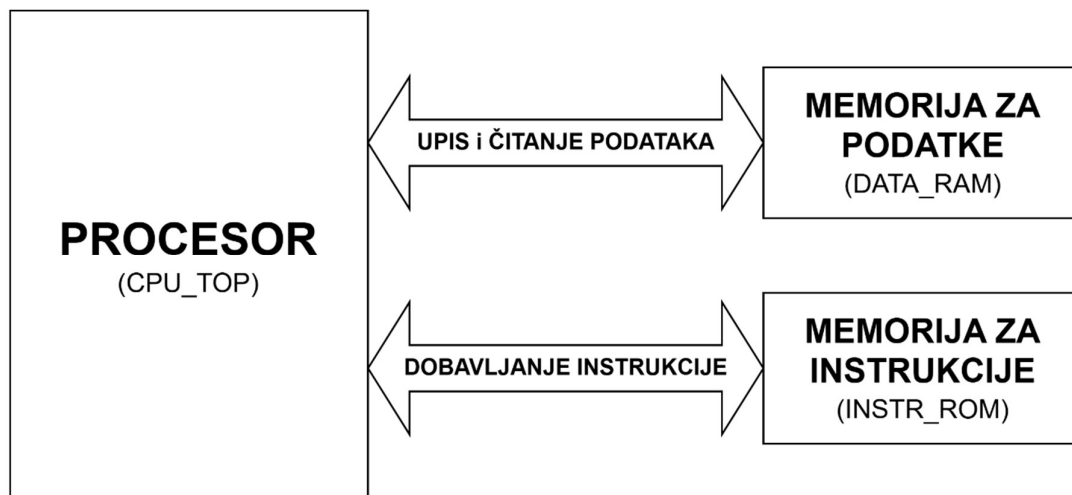
U zavisnosti od toga da li se koristi zajednička ili odvojena memorija za smeštanje podataka i instrukcija programa, razlikujemo dva osnovna tipa arhitekture:

- **Von Neumann arhitektura** – zajednička memorija za podatke i instrukcije.
- **Harvardska arhitektura** – ima posebne memorije za podatke i instrukcije.

2. Vrh hijerarhije računarskog sistema LPRS1

LPRS1 procesor koji ćemo projektovati na vežbama je **Harvardске arhitekture**, što znači da su memorije za podatke i instrukcije odvojene. Vrh hijerarhije ovog računarskog sistema je prikazan na Slici 2-1 i osnovne komponente su:

- **Procesor** (CPU_TOP) – izvršava program iz instrukcione memorije
- **Memorija za podatke** (DATA_RAM) – sadrži podatke koji će se koristiti u programu
- **Instrukciona memorija** (INSTR_ROM) – sadrži niz instrukcija (program) koje treba da se izvrše

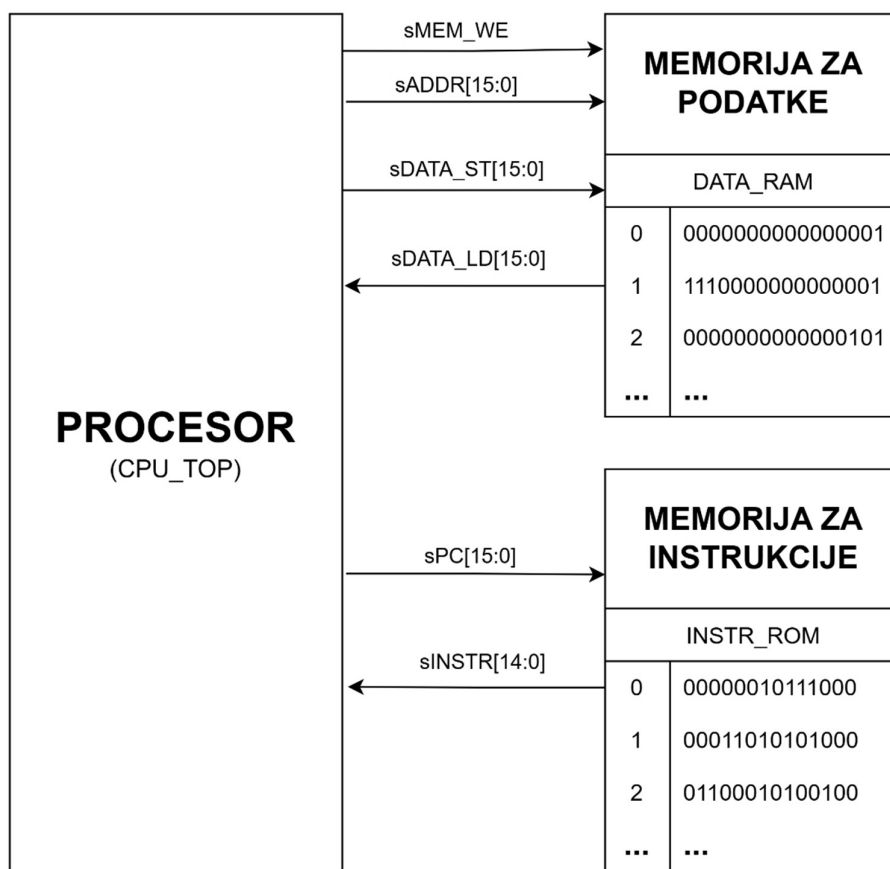


Slika 2-1 – Vrh hijerarhije sistema

Memorija za instrukcije (INSTR_ROM) čuva niz **instrukcija** (PROGRAM) koje je potrebno izvršiti, dok **memorija za podatke (DATA_RAM)** čuva **podatke**. Procesor komunicira sa memorijom za instrukcije kako bi dobio instrukciju koju treba da izvrši, dok sa memorijom za podatke komunicira kada treba da pročita ili upiše neki podatak/rezultat u memoriju za podatke.

Procesor u sebi ima **programski brojač PC** (eng. *Program Counter*) koji sadrži redni broj instrukcije koju procesor treba da dobavi i izvrši. Izlaz iz programskog brojača je signal **sPC[15:0]** i procesor ovim signalom zahteva dobavljanje tačno određene instrukcije iz instrukcione memorije.

Instrukciona memorija koristi vrednost programskog brojača **sPC** kao adresu instrukcije. Čita instrukciju sa zadate adrese **sPC** i šalje je procesoru preko signala **sINSTR[14:0]**.



Slika 2-2 – Komunikacija procesora sa memorijama za podatke i instrukcije

Memorija za podatke (DATA_RAM) se koristi za čuvanje podataka koji mogu da se upisuju ili čitaju iz memorije u zavisnosti od zadate instrukcije koju treba da izvrši procesor.

Ukoliko je potrebno izvršiti upis podataka u memoriju, instrukcija u sebi sadrži podatak koji je potrebno upisati **sDATA_ST[15:0]**, kao i adresu za upis podatka **sADDR[15:0]**. Pored toga se obavezno šalje i aktivan signal za dozvolu upisa **sMEM_WE**.

Ukoliko je potrebno pročitati podatak iz memorije, instrukcija sadrži adresu za čitanje podatka **sADDR[15:0]** sa koje treba pročitati podatak, koji se zatim šalje procesoru preko signala **sDATA_LD[15:0]**.

2.1. Program i format instrukcije

Program koji treba da se izvrši na procesoru predstavlja **niz instrukcija** koji se nalazi u instrukcionoj memoriji.

LPRS1 procesor podržava tri vrste instrukcija:

1. **Instrukcije za računanje** – aritmetičke i logičke operacije (rad sa registrima)

ADD R3, R2, R1 → R3 = R2 + R1

2. **Instrukcije za čitanje i pisanje podataka** – pristupaju podacima u memoriji za podatke

LD R1, R2 → R1 = DATA_MEM[R2]

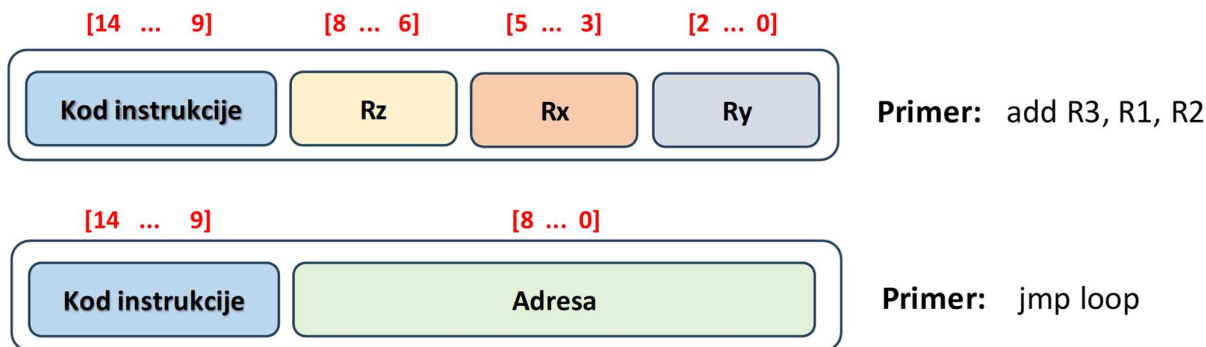
3. **Instrukcije skoka/grananja** – menjaju vrednost programskog brojača

JMP end → PC = addr[end]

Svaka instrukcija je kodovana sa **15 bita**. Format instrukcije zavisi od tipa instrukcije i može biti u jednom od formata koji su prikazani na Slici 2-3:

1. **IIIIIZZXXYYY**
2. **IIIIIAAAAAA**

Format instrukcije zavisi od toga da li instrukcija treba nešto da izračuna ili da menja adresu (skok u programu). Oba formata instrukcije sadrže na početku kôd instrukcije širine 6 bita označen sa **IIIIII**. U prvom formatu je sa **ZZZ** označena adresa odredišnog registra **Rz** gde treba upisati rezultat, sa **XXX** registar **Rx** (operand 1), a sa **YYY** registar **Ry** (operand 2). Ukoliko se koristi drugi format instrukcije, 9-bitni deo **AAAAA** predstavlja adresu skoka. Kôd instrukcije označen sa **IIIIII** se može predstaviti sa **TT0000**, gde **TT** označava tip instrukcije, a **0000** označava kôd operacije, koji će dodatno biti opisan u opisu instrukcija.

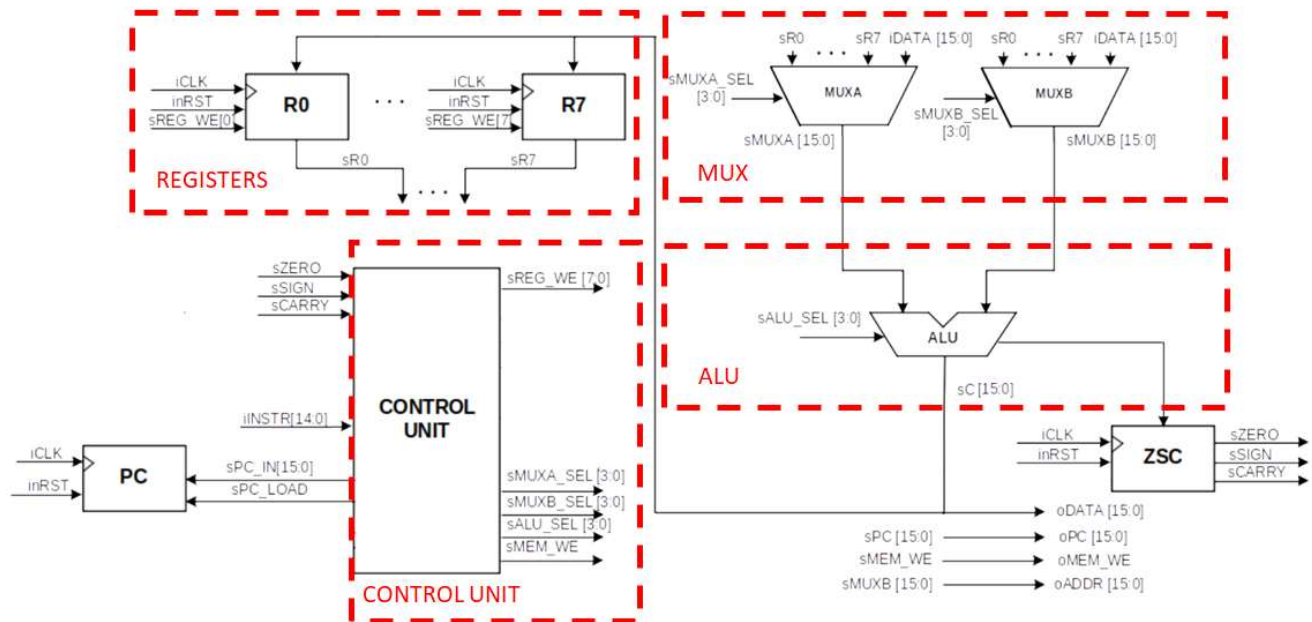


Slika 2-3 – Formati instrukcije

3. Arhitektura LPRS1 procesora

LPRS1 procesor je **16-bitna arhitektura** sa 8 registara opšte namene koji su širine 16 bita. Svaka instrukcija je kodovana sa 15 bita, a sve celobrojne operacije se obavljaju između registara, sa najviše dva izvorna operanda i jednim odredišnim registrom.

Na Slici 3-1 je prikazana arhitektura LPRS1 procesora (CPU_TOP) koji je specijalno dizajniran za potrebe računarskih vežbi i koga je potrebno implementirati zajedno sa njegovim podmodulima.



Slika 3-1 – Arhitektura LPRS1 procesora

Moduli su posebno obeleženi na slici i obuhvataju sledeće:

1. **Registri** – 8 registara opšte namene širine 16 bita
2. **Multiplekseri** – izbor operandada nad kojima će se izvršiti zadata instrukcija
3. **Aritmetičko-logička jedinica** – ALU – izvršava aritmetičke i logičke operacije
4. **Upravljačka jedinica** – CU – upravlja kompletnim sistemom
5. **Programski brojač** – PC – za dobavljanje instrukcije koja treba da se izvrši na procesoru.

Svaka komponenta je opisana u zasebnoj vhd datoteci, a povezane su u vrhu hijerarhije sistema **cpu_top.vhd**.

Kako se izvršava jedna instrukcija u procesoru?

1. **PC → Instrukciona memorija:** Instrukcija se učitava iz instrukcione memorije sa adrese koju određuje programski brojač **sPC**.
2. **Dekodovanje:** Instrukcija se šalje upravljačkoj jedinici (CU) koja dekoduje instrukciju **sINSTR[14:0]** i na osnovu nje definiše upravljačke signale za svaku komponentu procesora.
3. **Izbor operandada:** Na osnovu instrukcije se dobavljaju operandi iz registara
4. **ALU operacija:** Dobavljeni operandi se usmeravaju ka ALU da izvrši navedenu operaciju
5. **Upis rezultata:** u odredišni registar ili u memoriju.
6. **Ažuriranje PC:** kako bi se postavila adresa sledeće instrukcije.

PRIMER: Izvršavanje instrukcije za sabiranje 2 broja: $c = a + b$

$c = a + b$	=>	ADD R3, R2, R1	=>	000001011010001
-------------	----	-----------------------	----	------------------------

Upravljačka jedinica će iz instrukcione memorije dobiti vrednost signala ***sINSTR***[14:0] <= „000001011010001“ i na osnovu te vrednosti treba da generiše vrednosti upravljačkih signala:

sREG_WE [7:0] = „00001000“	-- registar u koji treba upisati rezultat je R3 , pa je 3. bit aktivan
sMUXA_SEL [3:0] = „0010“	-- operand A je R2
sMUXB_SEL [3:0] = „0001“	-- operand B je R1
sALU_SEL [3:0] = „0001“	-- Kod operacije sabiranja je 0001
sMEM_WE = '0'	-- Rezultat se ne smešta u memoriju za podatke

Na osnovu ovih definisanih vrednosti će svaka od komponenti znati šta treba da radi:

- MUX_A će selektovati registar R2 kao operand A
- MUX_B će selektovati registar R1 kao operand B
- ALU jedinica će izvršiti operaciju sabiranja
- U registar R3 će se upisati signal **sC** koji je rezultat zadate operacije iz ALU jedinice.

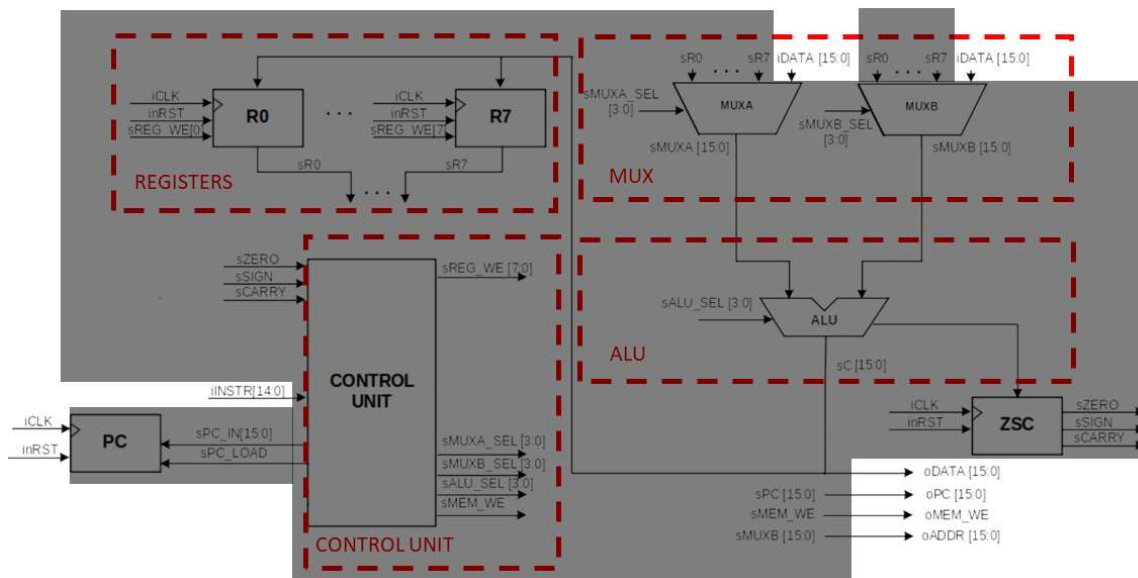
4. Opis vrha hijerarhije sistema u VHDL jeziku

Osnovni principi projektovanja složenih sistema, ne samo u digitalnom svetu, su principi apstrakcije, modularnosti i skrivanja informacija, koji omogućavaju da se neki deo sistema može „zatvoriti“ u crnu kutiju i koristiti samo posmatrajući njegove ulaze i izlaze. Složeni sistemi se projektuju kombinovanjem ovih crnih kutija povezivanjem njihovih prolaza. Koristeći ovaj pristup, sistem se može posmatrati kroz nivoe apstrakcije – od najnižih (digitalna logička kola, flip-flopovi) do najviših (**vrh hijerarhije sistema**).

Princip skrivanja informacija znači da, nakon što deo sistema „zatvorimo“ u crnu kutiju, korišćenje tog dela sistema ne sme da zavisi od njegovog načina realizacije, odnosno od onoga što se nalazi u crnoj kutiji. Ovaj princip olakšava rad korisniku, jer *ne mora* da poznaje arhitekturu sistema ako želi da ga iskoristi kao komponentu, ali i obavezuje autora modula da ga napravi potpunog, iskoristljivog isključivo putem svojih prolaza i specifikacije. Isto tako u VHDL jeziku, kada završite projektovanje vašeg *entiteta (entity)*, on mora biti iskoristljiv kroz svoje *prolaze (ports)* i specifikaciju ponašanja za date ulaze.

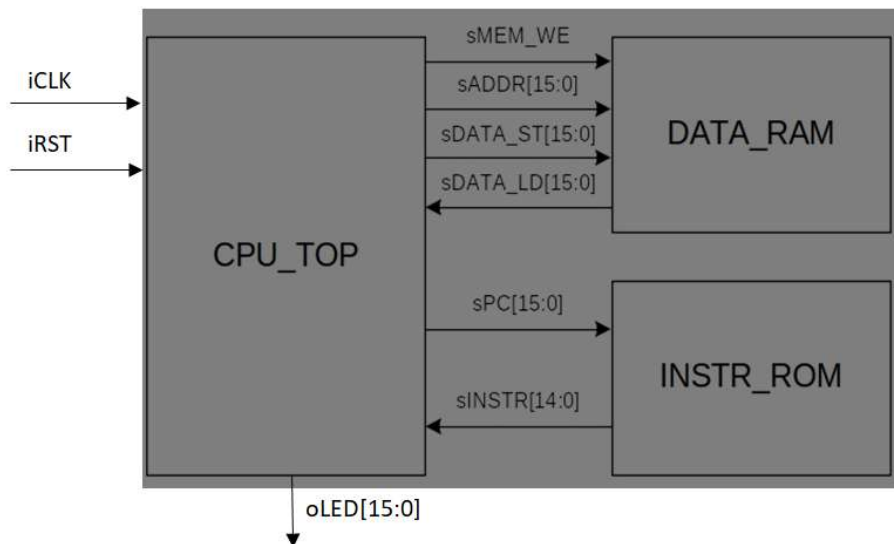
Slično možemo da uradimo i sa gotovim sistemom na primeru procesora. Kao što je prikazano na Slici 4-1, mi taj sistem možemo „zatvoriti“ u crnu kutiju i koristiti ga samo preko njegovih ulaza/izlaza.

Na višem nivou apstrakcije, možemo da **instanciramo** neku komponentu sistema i povežemo je na ostatak sistema. U terminologiji digitalnih sistema, opis sistema na najvišem nivou hijerarhije se naziva **top level**. Na slici 4-1 je prikazan procesor zatvoren u crnu kutiju i ovaj deo sistema je opisan u datoteci ***cpu_top.vhd***.



Slika 4-1. LPRS1 procesor zatvoren u crnu kutiju (CPU TOP)

Kako bismo prešli na još viši nivo apstrakcije, napravljena je i datoteka **top.vhd** u kojoj je instanciran CPU TOP modul, kao i memorije za podatke i instrukcije, koje su uvezane u vrh hijerarhije sistema tj. TOP modul.

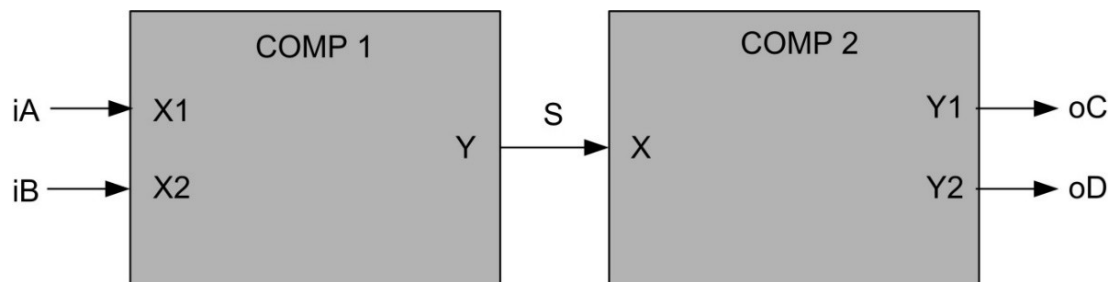


Slika 4-2. Top level procesora

Kako bi pokazali kako se vrši mapiranje komponenti, posmatrajmo top level sistema na Slici 4-3, koji se sastoji od dve komponente COMP1 i COMP2. Neka komponenta COMP1 ima dva ulaza (X1, X2) i jedan izlaz (Y). Neka komponenta COMP2 ima jedan ulaz (X) i dva izlaza (Y1, Y2). Komponente treba povezati kao na Slici 4-3. Komponente COMP1 i COMP2 su definisane kao posebni entiteti, a **njihovi opisi se nalaze u posebnim VHDL datotekama**.

Listing 4-1 prikazuje arhitekturu najvišeg nivoa apstrakcije sistema (TOP LEVEL) gde se vidi način povezivanja komponenti u VHDL jeziku. Sistem ima dva ulaza (iA, iB) i dva izlaza (oC, oD) koji treba da se povežu na ulazne komponente COMP1 i izlazne komponente COMP2, respektivno. Izlaz komponente COMP1 povezati sa ulazom

komponente COMP2. Komponente se međusobno vezuju žicama, odnosno **internim signalima** u VHDL-u. U ovom primeru, neka to bude signal S.



Slika 4-3. Primer sistema sa dve komponente

Listing 4-1. Arhitektura sistema sa Slike 4-3

```

entity MyTopLevel is port (
    iA    : in std_logic;
    iB    : in std_logic;
    oC    : out std_logic;
    oD    : out std_logic);
end entity;

architecture Behavioral of MyTopLevel is

    signal S : std_logic;

begin
    iCOMP1 : entity work.COMP1 port map (
        X1 => iA,
        X2 => iB,
        Y  => S
    );
    iCOMP2 : entity work.COMP2 port map (
        X  => S,
        Y1 => oC,
        Y2 => oD
    );
end Behavioral;
  
```

Mapiranje komponente zahteva sledeće korake:

- Potrebno je dodati VHDL datoteku u kojoj je opisana komponenta u projekat (izborom **Project --> Add/Remove Files in Project...**)
- Nakon ključne reči **begin**, komponentu treba mapirati koristeći sintaksu iz Listinga 4-1. Prilikom navođenja veza, ime pre operatora obrnute dodele (=>) je naziv **prolaza komponente**, a ime nakon njega je **naziv signala u vrhu hijerarhije** koji se povezuje na odgovarajući prolaz.

5. Komponente LPRS1 procesora

5.1. Programski brojač PC

Programski brojač je širine 16 bita i sadrži adresu trenutne instrukcije iz instrukcione memorije. Vrednost brojača se ažurira nakon svake izvršene instrukcije dodavanjem vrednosti 1 njegovoj trenutnoj vrednosti. Naravno ovakvo sekvencijalno izvršavanje i povećavanje adrese za 1 bi bilo dovoljno da ne postoje petlje, skokovi, uslovi, pozivi procedura, itd. Tako da promena vrednosti programskog brojača zavisi od načina skoka. Ukoliko imamo **uslovne skokove** i ako je uslov zadovoljen, programski brojač se menja i izvršavanje se postavlja na **novu lokaciju**. Ako uslov nije ispunjen, programski brojač se jednostavno uvećava za 1 i izvršavanje se nastavlja sa instrukcijom koja je sledila iza instrukcije skoka.

Programski brojač je realizovan u datoteci **cnt.vhd** i takođe je instanciran i uvezan u vrhu hijerarhije sistema u datoteci **cpu_top.vhd**.

5.2. Registarska logika

LPRS1 procesor ima 8 registara širine 16 bita koji čuvaju međurezultate, izlaze iz komponenti, itd. na putu od ulaza do izlaza sistema.

Registar je sekvencijalna komponenta koja na ulazu prihvata 16-bitni podatak koji dolazi iz ALU jedinice (rezultat ALU operacije). Ovaj podatak će se upisati u registar ako je signal dozvole upisa **sREG_WE** aktivan. Izlazi iz registara se ponovo šalju na ulaz ALU jedinice kao operandi A i B.

Zašto su registri odvojeni od memorije za podatke?

Savremeni programi su poprilično veliki, tako da je poželjno imati veliku memoriju koja će to podržati. Ali sa druge strane, velike memorije su spore i obično podržavaju samo pristup jednoj lokaciji u trenutku. S druge strane, ako koristimo registre za čuvanje operanada ili služe kao odredište, možemo projektovati hardver registra za paralelni pristup i učiniti ga veoma brzim. Da bismo zadržali brzinu, nećemo moći da imamo mnogo registara – što je klasično poređenje performansi veličine i brzine. Kompromis koji vodi do najboljih performansi je imati mali broj veoma brzih registara koje koristi većina instrukcija i velika, ali spora glavna memorija.

Registar je opisan u datoteci **register.vhd** i s obzirom na to da imamo ukupno 8 registara opšte namene, povezaćemo 8 registarskih komponenti u posebnoj datoteci za opis vrha hijerarhije sistema u datoteci **cpu_top.vhd**.

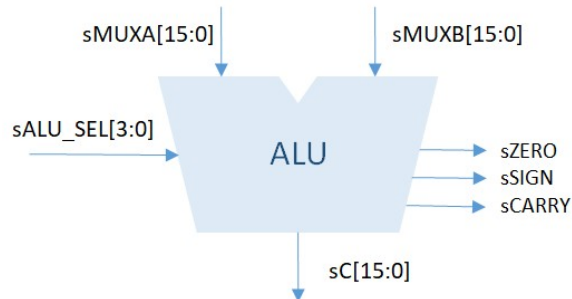
5.3. Multiplekseri za izbor operanada

Izlazni signali iz registara R0-R7 se dovode na multipleksere A i B, koji na osnovu selekcionih signala **sMUXA_SEL** i **sMUXB_SEL** prosleđuju jedan od 9 ulaza ALU jedinici kao operande A i B. Pored vrednosti registara R0-R8, moguće je kao operand koristiti i učitano vrednost iz memorije za podatke **iDATA**.

Multiplekser je realizovan u datoteci **mux.vhd** i s obzirom na to da u sistemu imamo 2 multipleksere, u top modulu **cpu_top.vhd** će biti instancirane i povezane 2 komponente za MUXA i MUXB.

5.4. Aritmetičko-logička jedinica (ALU)

Aritmetičko-logička jedinica izvršava aritmetičke i logičke operacije nad dva 16-bitna ulazna operanda **A** i **B**. Operacija koja će se izvršiti se definiše pomoću signala **sALU_SEL**, dok su izlazi iz sistema 16-bitni rezultat **sC**, kao i jednobitni statusni biti **sZERO**, **sSIGN** i **sCARRY**.



Slika 3-1 – Aritmetičko-logička jedinica

Operacije koje su podržane u procesoru i koje će se izvršavati u ALU jedinici prikazane su u Tabeli 3-1. U prvoj koloni tabele su navedene operacije i nazivi instrukcija sa operandima, u drugoj koloni funkcije svake instrukcije, zatim sledi kôd instrukcije i u poslednjoj koloni je opis svake instrukcije.

Tabela 3-1. Instrukcije procesora

Operacija	Funkcija	Kôd instrukcije	Opis instrukcije
mov Rz, Rx	$RZ \leftarrow [RX]$	00 0000	Prebacuje sadržaj registra X u registar Z
add Rz, Rx, Ry	$RZ \leftarrow [RX] + [RY]$	00 0001	Zbir registara X i Y upisuje u registar Z
sub Rz, Rx, Ry	$RZ \leftarrow [RX] - [RY]$	00 0010	Razliku registara X i Y upisuje u registar Z
and Rz, Rx, Ry	$RZ \leftarrow [RX] \& [RY]$	00 0011	Rezultat logičkog I registara X i Y upisuje u registar Z
or Rz, Rx, Ry	$RZ \leftarrow [RX] \mid [RY]$	00 0100	Rezultat logičkog ILI registara X i Y upisuje u registar Z
not Rz, Rx	$RZ \leftarrow \text{not } [RX]$	00 0101	Negiranu vrednost registra X upisuje u registar Z
inc Rz, Rx	$RZ \leftarrow [RX] + 1$	00 0110	Inkrementira registar X i upisuje u registar Z
dec Rz, Rx	$RZ \leftarrow [RX] - 1$	00 0111	Dekrementira registar X i upisuje u registar Z
shl Rz, Rx	$RZ \leftarrow \text{shl } [RX]$	00 1000	Logički pomeren registar X ulevo upisuje u registar Z
shr Rz, Rx	$RZ \leftarrow \text{shr } [RX]$	00 1001	Logički pomeren registar X udesno upisuje u registar Z
ashl Rz, Rx	$RZ \leftarrow \text{ashl } [RX]$	00 1010	Aritmetički pomeren registar X ulevo upisuje u registar Z
ashr Rz, Rx	$RZ \leftarrow \text{ashr } [RX]$	00 1011	Aritmetički pomeren registar X udesno upisuje u registar Z

U tabeli 3-1 možete primetiti da je tip instrukcije za sve ALU operacije kodovan sa najviša 2 bita vrednosti **00**, a u nastavku sledi 4-bitan **kôd operacije** na osnovu koga će se selektovati operacija koju treba izvršiti.

Statusni biti koje aritmetičko logička jedinica određuje su:

- **ZERO** – pokazuje da li je rezultat operacije jednak NULI (**sZERO** = 1 ako je rezultat jednak NULI)
- **SIGN** – pokazuje da li je rezultat pozitivan ili negativan (**sSIGN** = 1 – negativan, **sSIGN** = 0 – pozitivan)
- **CARRY** – pokazuje da li ima prenosa nakon aritmetičke operacije (**sCARRY** = 1 ako se desio prenos)

U datoteci **alu.vhd** treba opisati aritmetičko-logičku jedinicu LPRS1 procesora i povezati u **cpu_top.vhd**. S obzirom na to da je potrebno realizovati prenos, proširite operande sa dodatnim bitom.

5.5. Upravljačka jedinica

Upravljačka jedinica treba da generiše upravljačke signale (selekcione i signale dozvole upisa), za ostale delove digitalnog sistema kao što su:

- ***sREG_WE[7:0]*** – dozvola upisa u registre opšte namene R0-R7
- ***sPC_IN*** i ***sPC_LOAD*** – upravljački signali programskog brojača
- ***sMUXA_SEL[3:0]*** i ***sMUXB_SEL[3:0]*** – selekcioni signali za izbor prvog i drugog operanda
- ***sALU_SEL*** – kôd operacije aritmetičko-logičke jedinice
- ***sMEM_WE*** – dozvola upisa u memoriju za podatke

Upravljačka jedinica definiše svoje izlaze na osnovu trenutno izvršavane instrukcije i statusnih bita iz ALU jedinice. U datoteci ***control_unit.vhd*** treba opisati upravljačku jedinicu LPRS1 procesora.

5.6. Pristup memoriji za podatke

Pristup memoriji za podatke je omogućen preko instrukcija **LOAD** i **STORE** koje su opisane u Tabeli 3-2.

Tabela 3-2. Instrukcije za pristup memoriji za podatke

Operacija	Funkcija	Kôd instrukcije	Opis
ld R _z , R _y	$RZ \leftarrow [[RY]]$	10 0000	Upisuje u registar RZ podatke koji se nalaze na adresi koja je upisana u registar RY
st R _x , R _y	$[[RY]] \leftarrow [RX]$	11 0000	Upisuje podatak iz registra RX u memoriju za podatke na adresi koja se nalazi u registru RY

Format ovih instrukcija je sličan kao i kod aritmetičko-logičkih, s tom razlikom da se ignorišu neka polja instrukcije, kao na primer polje koda operacije (0000). Polje tipa operacije određuje da li je u pitanju **LOAD** instrukcija (vrednost 10) ili pak **STORE** instrukcija (vrednost 11).

Instrukcija **LOAD** upisuje u registar **RZ** sadržaj iz memorije za podatke sa adrese koja se nalazi u registru **RY** (zbog toga je u opisu funkcije stavljeno dvostruko dereferenciranje registra **RY**). Izvršavanje instrukcije se odvija u sledećem toku:

- Multiplexer ***MUXB*** selektuje registar ***RY*** i njegova vrednost se prosleđuje na izlaz ***oADDR***.
- Vrednost iz memorije se pojavljuje na ulazu ***iDATA***.
- Multiplexer ***MUXA*** selektuje vrednost ***iDATA***.
- ALU propušta vrednost iz registra ***A*** koja je jednaka vrednosti ***iDATA***.
- Vrednost ***iDATA*** se upisuje u registar ***RZ***.

Instrukcija **STORE** upisuje u memoriju za podatke, na adresu koja se nalazi u registru ***RY***, vrednost koja se nalazi u registru ***RX***. Izvršavanje instrukcije se odvija u sledećem toku:

- Multiplexer ***MUXB*** selektuje registar ***RY*** i njegova vrednost se prosleđuje na izlaz ***oADDR***.
- Multiplexer ***MUXA*** selektuje registar ***RX***.
- ALU propušta vrednost iz registra ***RX***.
- Izlaz iz ALU se prosleđuje na memoriju preko signala ***oDATA***.
- Postavlja se signal dozvole ***MEM_WE*** i time dovršava upis u memoriju.

5.7. Instrukcije skoka – bezuslovni skok

Instrukcija bezuslovnog skoka je prikazana u Tabeli 3-3. Ova instrukcija ima sledeći format: **IIIIIAAAAAAAAA** i sadrži kod instrukcije i adresu skoka. Polje tipa instrukcije **TT** je kodovano sa **01**.

Tabela 3-3. Instrukcija bezuslovnog skoka

Operacija	Funkcija	Kod instrukcije
jmp ADDR	$PC \leftarrow ADDR$	01 0000

Adresa na koju se skače treba da se upiše u programski brojač. U tu svrhu se koriste signali **sPC_IN** i **sPC_LOAD**. Pošto se rezultat operacije ne upisuje u registre opšte namene, signal **oREG_WE** je neaktivan tokom instrukcije **JMP**, pa se vrednost ne upisuje ni u jedan registar procesora.

5.8. Instrukcije skoka – uslovni skok

Instrukcije uslovnog skoka su date u Tabeli 3-6-1. Ove instrukcije imaju isti format kao instrukcija bezuslovnog skoka, sa drugačijim kodom instrukcije. Polje operacije definišu statusni biti **sZERO**, **sSIGN** i **sCARRY** koji su označeni sa **Z**, **S** i **C** respektivno. Instrukcija uslovnog skoka se takođe sastoji iz 6-bitnog koda instrukcije i 9-bitne adrese na koju se skače.

Tabela 3-6-1. Instrukcije uslovnog skoka

Operacija	Funkcija	Kod instrukcije
jmpz ADDR	if $Z=1$ $PC \leftarrow ADDR$	01 0001
jmps ADDR	if $S=1$ $PC \leftarrow ADDR$	01 0010
jmpc ADDR	if $C=1$ $PC \leftarrow ADDR$	01 0011
jmpnz ADDR	if $Z=0$ $PC \leftarrow ADDR$	01 0101
jmpns ADDR	if $S=0$ $PC \leftarrow ADDR$	01 0110
jmpnc ADDR	if $C=0$ $PC \leftarrow ADDR$	01 0111

Kao i kod instrukcije bezuslovnog skoka, ponašanje **oREG_WE** je isto kao i kod instrukcije bezuslovnog skoka.

ZADACI

1. Projektovanje procesora

Projekat u kome je opisan LPRS1 procesor se sastoji iz više vhd datoteka, tačnije svaka zasebna komponenta je opisana u zasebnoj vhd datoteci, gde su neke komponente usled trivijalnosti već realizovane. U datoteci **alu.vhd** je potrebno realizovati aritmetičko-logičku jedinicu (ALU) procesora, dok je u datoteci **control_unit.vhd** potrebno realizovati upravljačku jedinicu (CU). ALU treba da računa rezultat operacije nad svojim ulaznim operandima, u zavisnosti od selekcionih bita. Prolazi ALU jedinice su prikazani u Tabeli 5, dok su podržane operacije sumirane u Tabeli 6. ALU treba da bude implementirana kao **kombinaciona mreža**. Pretpostaviti da sve operacije rade sa označenim brojevima u II komplementu.

Tabela 5. Prolazi ALU

Prolaz	Smer	Funkcija
iA [15:0]	in	prvi ulazni operand
iB [15:0]	in	drugi ulazni operand
iSEL [3:0]	in	selekcija operacije
oC [15:0]	out	rezultat operacije
oZERO	out	statusni bit, rezultat jednak nuli
oSIGN	out	statusni bit, rezultat negativan
oCARRY	out	statusni bit, rezultat ima prenos

Tabela 6. Operacije ALU

Kod operacije	Operacija
0000	A
0001	A + B
0010	A – B
0011	A and B
0100	A or B
0101	not (A)
0110	A + 1
0111	A – 1
1000	shl (A)
1001	shr (A)
1010	ashl (A)
1011	ashr (A)

Statusni biti takođe treba da budu realizovani kombinaciono, odnosno kao funkcije rezultata iz ALU. Aktivna vrednost statusnih bita treba da bude na visokom nivou.

U upravljačkoj jedinici je potrebno opisati kombinacione mreže, mahom dekodere, komparatore i multipleksere koji će odrediti upravljačke signale.

Radi lakše realizacije ALU i CU, instrukcije će biti dodavane postepeno i biće proveravane putem simulacije. Radi testiranja se koristi algoritam kumulativnog sabiranja datog u datoteci **cumsum.asm**.

Za početak je potrebno implementirati prvu instrukciju **inc** u ALU, kao i dodelu vrednosti sledećim upravljačkim signalima u CU: **MUXA_SEL**, **ALU_SEL**, **REG_WE**. Nakon toga je potrebno pokrenuti simulaciju. Uveriti se da se u registru R0 nakon rastuće ivice takta tj. izvršenja intrukcije **inc** nalazi vrednost 1. Ukoliko se ne nalazi, proveriti da li su svi signali bili na očekivanim vrednostima.

Dodatak: Asembliranje

Asembliranje je prevođenje programa napisanog u asemblerskom jeziku u mašinski format odnosno u odgovarajući format instrukcija koje će izvršiti procesor, u ovom slučaju LPRS1 procesor.

Sve programi napisani u assembleru moraju da se nalaze u direktorijumu **src/asm**.

Ukoliko je potrebno prevesti i izvršiti neki drugi program napisan u asemblerskom jeziku, potrebno je izmeniti skriptu za prevođenje koja se nalazi u glavnom direktorijumu projekta i naziva se **asm.tcl**. Dozvoljeno je menjati samo **naziv datoteke** koja sadrži program napisan u assembleru:

```

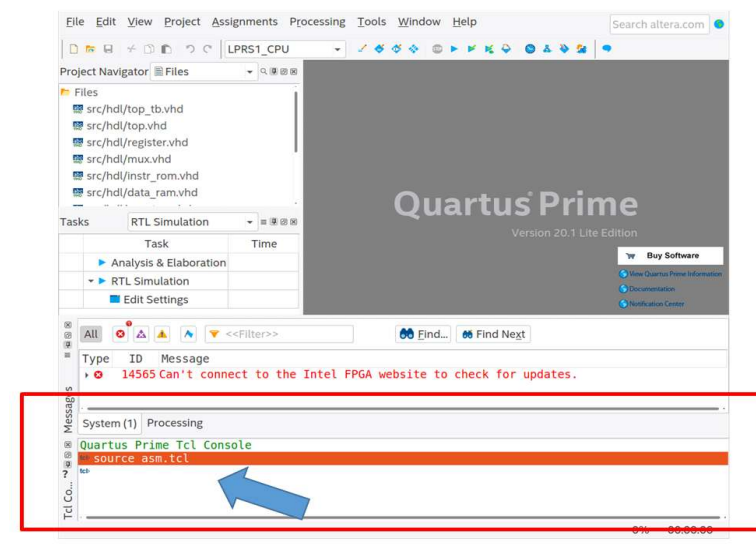
1  #!/usr/bin/env tcl
2  #####
3  # LPRS1 assembler script.
4  # Author: Milos Subotic <milos.subotic@uns.ac.rs>
5  #####
6
7  set asm_file "src/asm/cumsum.asm.txt"
8
9  #####
10
11 set OS [lindex $tcl_platform(os) 0]
12 set result f

```

Za asembliranje, otvoriti „Tcl Console“ korišćenjem prečice **Alt+2** i zadati komandu praćenu enterom:

```
source asm.tcl
```

Slika 6 prikazuje izgled Quartus okruženja prilikom asembliranja.



Slika 6 - Asembliranje u Quartusu

Nakon asembliranja, promeniće se sadržaj sledećih vhdL datoteka: ***data_ram.vhdl*** i ***instruction_rom.vhdl***. Ukoliko su ove datoteke bile otvorene, potrebno je prihvatiti izmene nakon asembliranja.

PITANJE: Zašto su ove dve datoteke izmenjene i koje podatke sadrže iz cumsum.asm?
--

S obzirom na to da su promenjene vhdL datoteke, neophodno je i prevesti ceo Quartus projekat koji opisuje procesor.

NAPOMENA: Svaka promena u datoteci koja sadrži asemblerski program zahteva i ponovno asembliranje, a nakon toga i prevođenje celog projekta u Quartusu.

ZAKLJUČAK

Upravo ste projektovali LPRS1 procesor i sada bi već trebali da imate jasno razumevanje kako rade procesori i kako izvršavaju programe. U ovoj vežbi ste naučili kako da opišete ALU i upravljačku jedinicu LPRS1 procesora u VHDL-u, a zatim i videli na koji način možete testirati i simulirati ceo sistem.